This class returns a set of objects representing each Sidekiq process. If the set is empty…you have problems.

The Rake task will be `production:sidekiq:processes_check`, located in `lib/tasks/production/sidekiq/processes_check.rake`. It will check the `size` of the Sidekiq process set and, if it's 0, trigger the error catcher like we did in *Handling Permanent Failures via Monitoring*.

snapshots/4-1/sidekiq-book/lib/tasks/production/sidekiq/processes_check.rake

```
namespace :production do
  namespace :sidekiq do
    desc "Alert if Sidekiq isn't running"
    task processes_check: :environment do
      process_set = Sidekiq::ProcessSet.new
      if process_set.size == 0
        ErrorCatcherServiceWrapper.new.notify("Sidekiq is not running!")
      end
    end
  end
end
```

You can call this with `bin/rails production:sidekiq:processes_check`. Depending on how your app is hosted, you will need to set up something that calls it regularly. This command can be called frequently and—given what it's checking for, that's a good thing—you want to know as soon as possible if Sidekiq has stopped running.

If you don't have any sort of scheduling feature in your production environment, you can create a `bash` script to call this in a loop as a last resort:

```
#!/bin/sh

while true
do
  bin/rails production:sidekiq:processes_check
  sleep 5
done
```

(Hopefully, you have something more sophisticated and reliable.)